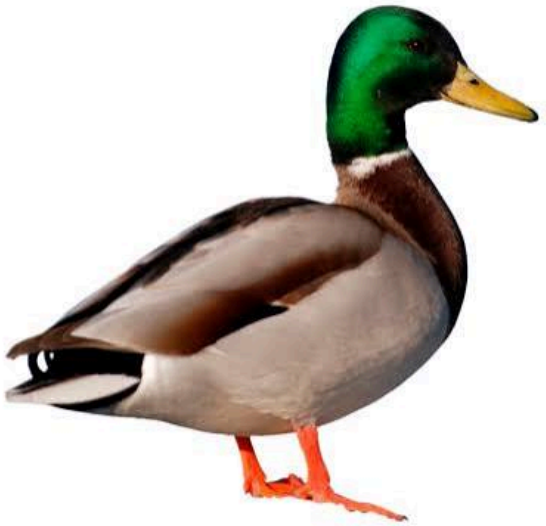
Ciência de Dados Aplicada ao Direito I

Aula 7

O que ficou da aula 6



Prof. Bruno da Cunha de Oliveira - 2026/2

Por que eu perguntei antes de explicar

Insper

Por que a pergunta vem antes

Existe evidência de que tentar resolver um problema antes de entendê-lo por inteiro faz você aprender mais.

Tentar é melhor do que esperar, e o palpite errado é o que faz a explicação seguinte ter em que pegar: ela corrige uma resposta que já é sua.

O que ficou da aula 6

Insper

Variável e valor

EXERCÍCIO

Nesta linha, qual par está na ordem (variável, valor)?

```
resultado = "PROCEDENTE"
```

A. `(resultado, "PROCEDENTE")`

B. `("PROCEDENTE", resultado)`

C. `(resultado, PROCEDENTE)`

D. `("resultado", "PROCEDENTE")`

Variável e valor

RESPOSTA

A. `(resultado, "PROCEDENTE").`

A variável é o nome estável do lugar onde registramos, e vem primeiro. O valor é o conteúdo guardado ali neste caso.

A C perde as aspas, e sem elas `PROCEDENTE` viraria o nome de outra variável. A D põe aspas no nome, e aí ele deixa de ser um nome e vira texto.

Variável, valor e tipo

```
resultado = "PROCEDENTE"
```

Elemento	Neste caso
variável	`resultado`, o nome estável do lugar onde registramos
valor	`"PROCEDENTE"`, o conteúdo registrado neste caso
tipo	a regra sobre quais valores cabem ali

Qual é o tipo? 1 de 10

84

Qual é o tipo? 1 de 10

RESPOSTA

Inteiro

Número sem parte fracionária. É a idade de um autor, e ninguém tem 84,5 anos para efeito de prioridade.

Qual é o tipo? 2 de 10

```
FALSO
```

Qual é o tipo? 2 de 10

RESPOSTA

Booleano

Duas respostas possíveis, e sempre as mesmas: VERDADEIRO ou FALSO. Em `tutela_deferida`, FALSO diz que houve pedido e ele foi indeferido.

Qual é o tipo? 3 de 10

```
12500.00
```

Qual é o tipo? 3 de 10

RESPOSTA

Ponto flutuante

Tem parte fracionária. Inteiro não teria, e como texto não daria para somar nem comparar por grandeza.

Qual é o tipo? 4 de 10

```
["LEI 9.656/98", "CDC"]
```

Qual é o tipo? 4 de 10

RESPOSTA

Lista

Colchete, itens separados por vírgula, ordem preservada. Uma decisão invoca vários fundamentos, e por isso o campo guarda vários valores.

Qual é o tipo? 5 de 10

```
"0000000-00.2026.8.26.0100"
```

Qual é o tipo? 5 de 10

RESPOSTA

Texto

Tem dígitos, mas não é número: ninguém soma dois números de processo. Os pontos e o hífen fazem parte do dado, e como número eles se perderiam.

Qual é o tipo? 6 de 10

```
01/09/2026
```

Qual é o tipo? 6 de 10

RESPOSTA

Data

Tipo próprio, e não texto. Só ele permite ordenar no tempo e calcular prazo. Como texto, `"02/01/2026"` viria antes de `"10/12/2025"`.

Qual é o tipo? 7 de 10

```
{"idade_autor": 67, "resultado": "PROCEDENTE"}
```

Qual é o tipo? 7 de 10

RESPOSTA

Dicionário

Abre e fecha com `{ }`, e cada valor tem uma chave. É pela chave que você pede o dado, e não pela ordem em que ele foi escrito.

Qual é o tipo? 8 de 10

```
Literal["DEFERIDA", "INDEFERIDA",
        "PREJUDICADA"]
```

Qual é o tipo? 8 de 10

RESPOSTA

Literal

Texto com conjunto fechado: aceita esses três valores e nenhum outro. Um quarto valor não é recusado por estar errado, e sim por não estar previsto.

Qual é o tipo? 9 de 10

```
data_transito = NULO
```

Qual é o tipo? 9 de 10

RESPOSTA

Nulo

Ausência do dado. O processo ainda não transitou, ou a decisão não informa. Não é o texto “NULO”, não é zero e não é FALSO.

Qual é o tipo? 10 de 10

"84"

Qual é o tipo? 10 de 10

RESPOSTA

Texto

As aspas mudam tudo. Na pergunta 1, `84` sem aspas era inteiro; aqui é uma sequência de dois caracteres. O tipo é decisão de quem registra, e não aparência do valor.

Chave e valor

EXERCÍCIO

Neste dicionário, qual par está na ordem (chave, valor)?

```
{"idade_autor": 67}
```

A. `("idade_autor", 67)`

B. `(67, "idade_autor")`

C. `("67", "idade_autor")`

D. `("idade_autor", "67")`

Chave e valor

RESPOSTA

A. `("idade_autor", 67).`

A chave vem primeiro e nomeia o campo. O valor vem depois e é o conteúdo guardado sob aquele nome.

A D acerta a ordem e erra o tipo: a idade é o inteiro `67`, e com aspas ela viraria o texto `"67"`, que é a pergunta 10 de novo.

Vocês já viram isto, mas ele tem nome

```
{"numero": "0000000-00", "idade_autor": 67, "resultado": "PROCEDENTE"}
```

chave

O nome que identifica o campo. É ela que você usa para pedir o dado. Em `"idade_autor": 67`, a chave é `"idade_autor"`.

valor

O conteúdo guardado sob aquela chave. Em `"idade_autor": 67`, o valor é `67`.

A chave vai entre aspas, porque ela é um texto. Repare no contraste com `idade_autor = 67`, em que a mesma palavra é o nome de uma variável e por isso vai sem aspas.

O que se repete

EXERCÍCIO

Numa lista em que cada item é o dicionário com os dados de uma decisão, o que se repete de um item para o outro?

A. os valores

B. as chaves

C. nada, cada decisão é diferente

D. o número do processo

O que se repete

RESPOSTA

B. As chaves.

```
[{"idade_autor": 67, "resultado": "PROCEDENTE"},
 {"idade_autor": 84, "resultado": "IMPROCEDENTE"}]
```

As chaves são as mesmas nos dois itens; só os valores mudam. É exatamente isso que torna a lista de dicionários uma tabela: todo item responde às mesmas perguntas.

Escrevam uma lista

Escrevam uma lista com três itens, do assunto que vocês quiserem: cidade, música, artigo de lei, o que for.

O assunto é livre. A notação não é.

Escrevam uma lista

RESPOSTA

```
["Salvador", "Recife", "Belém"]
```

Três coisas para conferir: `[` e `]` abrindo e fechando, vírgula entre os itens, e aspas em cada texto.

Nenhum item tem nome. Quem quiser o segundo pede pela ordem, e é só isso que a lista oferece.

Escrevam um dicionário

Agora escrevam um dicionário com três pares, sobre uma pessoa, um livro ou um processo.

Cada valor precisa da sua chave.

Escrevam um dicionário

RESPOSTA

```
{"titulo": "Grande Sertão: Veredas",
 "ano": 1956,
 "autor": "Guimarães Rosa"}
```

A conferência muda: `{` e `}` no lugar do colchete, dois-pontos entre a chave e o valor, e vírgula entre os pares.

As três chaves estão entre aspas, e isso é sempre assim: a chave é um texto. Nos valores é o tipo que decide, e por isso `1956` está sem aspas enquanto os outros dois estão com.

A notação distingue as duas estruturas

lista

```
["LEI 9.656/98",
 "SÚMULA 302 STJ"]
```

Abre e fecha com `[ ]`. Itens separados por vírgula. A ordem em que foram registrados fica preservada.

dicionário

```
{"idade_autor": 67,
 "resultado": "PROCEDENTE"}
```

Abre e fecha com `{ }`. Cada valor tem uma chave própria, e é por ela que você o pede.

Antes de coletar dados, alguém definiu isto

O que se define	Para o campo `resultado`
nome da variável	`resultado`
tipo	Literal de textos
valores permitidos	PROCEDENTE, IMPROCEDENTE, PARCIALMENTE PROCEDENTE, EXTINTO SEM RESOLUÇÃO DO MÉRITO
se a informação faltar	não se aplica: toda sentença tem um resultado

Esse documento tem nome

codebook

O conjunto das instruções de coleta, uma por campo.

Para cada campo: o nome, o tipo, os valores permitidos e o que fazer quando a informação não existe.

E os combinados que fazem duas pessoas codificarem o mesmo caso do mesmo jeito: de onde tirar cada informação e o que fazer com o caso duvidoso.

Escrito antes da coleta, é regra do que será feito.
Escrito depois, é descrição do que se fez.

Para que serve definir antes

Escrever isso antes custa algumas horas. O que essas horas compram:

- Você organiza os dados uma vez, em vez de reorganizar toda vez que precisa achar alguma coisa.

Para que serve definir antes

Escrever isso antes custa algumas horas. O que essas horas compram:

- Você organiza os dados uma vez, em vez de reorganizar toda vez que precisa achar alguma coisa.
- Você não descobre no fim que faltou um campo e tem que voltar às decisões para coletar tudo de novo.

Para que serve definir antes

Escrever isso antes custa algumas horas. O que essas horas compram:

- Você organiza os dados uma vez, em vez de reorganizar toda vez que precisa achar alguma coisa.
- Você não descobre no fim que faltou um campo e tem que voltar às decisões para coletar tudo de novo.
- A coleta já sai respondendo a pergunta que você quer responder, e não uma parecida.

Para que serve definir antes

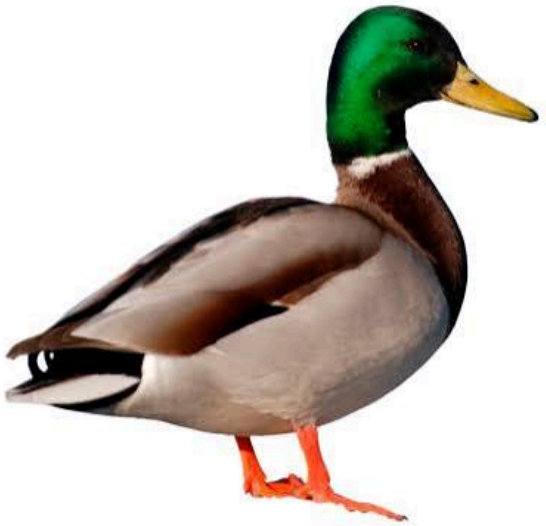
Escrever isso antes custa algumas horas. O que essas horas compram:

- Você organiza os dados uma vez, em vez de reorganizar toda vez que precisa achar alguma coisa.
- Você não descobre no fim que faltou um campo e tem que voltar às decisões para coletar tudo de novo.
- A coleta já sai respondendo a pergunta que você quer responder, e não uma parecida.
- Duas pessoas codificando a mesma decisão chegam ao mesmo registro. E uma das duas pode ser um LLM: sem o combinado escrito, ele decide sozinho, e decide diferente a cada vez que você pedir.

Ciência de Dados Aplicada ao Direito I

Aula 7

Condicionais, operadores e conectivos



Prof. Bruno da Cunha de Oliveira - 2026/2

Ordem das instruções

Insper

O que está guardado no fim

EXERCÍCIO

Depois destas duas linhas, o que está guardado em `prioridade`?

```
prioridade = "comum"
prioridade = "especial"
```

A. `"comum"`

B. `"especial"`

C. as duas, na ordem em que foram escritas

D. nada: a segunda linha não pode sobrescrever a primeira

O que está guardado no fim

RESPOSTA

B. `"especial"`. A última atribuição é a que vale.

Guardar não acumula: a segunda escrita apaga a primeira, e não sobra vestígio dela. Uma variável guarda um valor de cada vez.

É por isso que a ordem das linhas é conteúdo, e não arrumação.

A ordem das linhas é conteúdo

As instruções rodam de cima para baixo, uma de cada vez. Trocar duas de lugar pode mudar o que o programa faz.

```
prioridade = "comum"
prioridade = "especial"
```

A variável `prioridade` termina com o valor `"especial"` salvo.

```
prioridade = "especial"
prioridade = "comum"
```

A variável `prioridade` termina com o valor `"comum"` salvo.

Condicionais

Insper

Vocês já escreveram condicionais

Duas delas estavam na receita que a turma reconstruiu na aula 5:

1. “Se houver lacre de alumínio sob a tampa, puxe-o pela aba e descarte.”
2. “Se a Nutella acabar antes de a parte de dentro do pão ficar coberta, pare de passar Nutella e feche o sanduíche assim mesmo.”

Cada uma trata um caso que pode acontecer ou não. Falta só escrever de um jeito que uma máquina execute.

O pote não tem lacre

```
girar a tampa até sair
SE houver lacre sob a tampa ENTÃO
    puxar o lacre pela aba e descartar
```

Caso: o pote não tem lacre. Quais linhas rodam?

O pote não tem lacre

RESPOSTA

Só a primeira. A tampa sai, e mais nada roda: o pote fica aberto.

A condicional não interrompe o resto: ela só decide se aquele pedaço específico é executado. Sem lacre, o pedaço é pulado.

O pote tem lacre

```
girar a tampa até sair
SE houver lacre sob a tampa ENTÃO
    puxar o lacre pela aba e descartar
recolher a Nutella com a faca
```

Caso: o pote tem lacre. Quais linhas rodam, e em que ordem?

O pote tem lacre

RESPOSTA

As três, na ordem em que estão escritas: girar a tampa, puxar o lacre, recolher a Nutella.

Girar e recolher rodariam de qualquer jeito, com lacre ou sem. Só a linha do meio dependia da condição, e é só ela que está dentro do SE.

A pergunta que decide o que entra: isto roda nos dois casos? Se sim, fica de fora.

Duas condicionais seguidas

```
SE a parte de dentro do pão estiver coberta ENTÃO
    parar de passar Nutella
SE a Nutella acabar ENTÃO
    parar de passar Nutella
```

Caso: a parte de dentro do pão não está coberta e a Nutella acabou. Quais linhas rodam?

Duas condicionais seguidas

RESPOSTA

Para de passar Nutella. A primeira condição é falsa, a segunda é verdadeira, e basta uma delas para a instrução ser executada.

São duas condicionais independentes, e cada uma é testada por conta própria. A segunda não sabe o que a primeira respondeu.

Condicional

Em Python: `if`

Uma pergunta que só admite VERDADEIRO ou FALSO, e um pedaço de instrução que só é executado quando a resposta é VERDADEIRO.

EXEMPLO

```
SE houver lacre sob a tampa ENTÃO
    puxar o lacre pela aba e descartar
```

Indentação

Insper

Indentação

Os espaços no começo da linha não são enfeite. São eles que declaram quais instruções pertencem à condição e quais rodam de qualquer jeito.

DENTRO DO SE

```
SE o autor tiver 60 anos ou mais ENTÃO
    esta linha está dentro
esta linha está fora
```

Não está chovendo

```
SE estiver chovendo ENTÃO
    pegar o guarda-chuva
    fechar a janela
sair de casa
```

Caso: não está chovendo. Quais linhas rodam?

Não está chovendo

RESPOSTA

Só a última. Você sai de casa sem guarda-chuva e com a janela aberta.

Fechar a janela ficou dentro do SE por causa do recuo, e quem escreveu talvez não quisesse isso. A indentação decidiu, não a intenção.

A idade do autor está no processo

```
SE faltar a idade do autor no processo ENTÃO
    procurar a idade na qualificação da inicial
    registrar a decisão na planilha
```

Caso: a idade do autor está no processo. Quais linhas rodam?

A idade do autor está no processo

RESPOSTA

Nenhuma. A decisão não é registrada.

`registrar a decisão na planilha` está indentado, então ele depende de a idade estar faltando. O processo que já tem a idade pula as duas linhas.

É o erro inverso do guarda-chuva: lá uma linha de fora foi parar dentro, aqui a linha que precisava rodar sempre ficou presa na condição. E o resultado é uma planilha com buraco exatamente nos processos que estavam completos.

Uma indentação de diferença

VERSÃO A

```
SE houver lacre ENTÃO
    remover o lacre
    abrir a tampa
```

VERSÃO B

```
SE houver lacre ENTÃO
    remover o lacre
abrir a tampa
```

Caso: o pote não tem lacre. O que roda em cada versão?

Uma indentação de diferença

RESPOSTA

VERSÃO A

Nada roda. O pote fica fechado, porque abrir a tampa também depende de haver lacre.

VERSÃO B

A tampa é aberta. Só a remoção do lacre dependia da condição.

Uma indentação de diferença, e o programa faz outra coisa. Aqui a indentação não é estilo.

Um SE dentro do outro

```
SE estiver chovendo ENTÃO
    fechar a janela
    SE eu tiver guarda-chuva ENTÃO
        pegar o guarda-chuva
sair de casa
```

Caso: está chovendo e eu não tenho guarda-chuva. Quais linhas rodam?

Um SE dentro do outro

RESPOSTA

Fechar a janela e sair de casa.

A condição de fora deu VERDADEIRO, então tudo que está no primeiro recuo roda. A de dentro deu FALSO, e só a linha dela é pulada.

São dois níveis de recuo, e cada um responde a uma pergunta diferente.

O recurso não foi conhecido

```
autuar o recurso
SE o recurso for conhecido ENTÃO
    examinar o mérito do recurso
    SE houver razão ao recorrente ENTÃO
        dar provimento ao recurso
publicar o acórdão
```

Caso: o recurso não é conhecido. Quais linhas rodam?

O recurso não foi conhecido

RESPOSTA

Autuar o recurso e publicar o acórdão.

Quando a condição de fora dá FALSO, o bloco inteiro é pulado, e a pergunta sobre a razão do recorrente nem chega a ser feita. Não é que ela deu FALSO: ela não foi feita.

Vocês já conhecem isso por outro nome: recurso não conhecido não tem o mérito examinado. O que o recuo faz aqui é dizer que a admissibilidade vem antes.

A parte pediu gratuidade

```
SE a parte pediu gratuidade ENTÃO
    analisar o pedido
    SE não houver prova de recursos suficientes ENTÃO
        deferir a gratuidade
prosseguir com o processo
```

Caso: a parte pediu gratuidade, e há prova de que ela tem recursos suficientes. Quais linhas rodam?

A parte pediu gratuidade

RESPOSTA

Analisar o pedido e prosseguir com o processo.

A pergunta de fora deu VERDADEIRO, então o pedido é analisado. A de dentro deu FALSO, e só a linha dela é pulada: a gratuidade não é deferida, e o processo segue.

É a gratuidade da justiça do art. 98 do CPC, e a estrutura é a mesma da chuva e do recurso. Três domínios diferentes, e o recuo decide a mesma coisa nos três.

Um tab ou quatro espaços

- São esses dois os recuos que vocês vão encontrar em quase todo código.

Um tab ou quatro espaços

- São esses dois os recuos que vocês vão encontrar em quase todo código.

- O que a máquina cobra não é o tamanho, e sim a consistência: dentro do mesmo bloco, todas as linhas recuam igual.

Um tab ou quatro espaços

- São esses dois os recuos que vocês vão encontrar em quase todo código.
- O que a máquina cobra não é o tamanho, e sim a consistência: dentro do mesmo bloco, todas as linhas recuam igual.
- Escolhido um dos dois, ele vale para o arquivo inteiro, e não só para o bloco em que você está.

Misturar tab e espaço no mesmo bloco é erro, mesmo quando o resultado parece alinhado na tela.

Operadores de comparação

Insper

Isto é condição?

EXERCÍCIO

Qual destas é uma condição, no sentido da aula?

A. `idade_autor = 67`

B. `idade_autor >= 60`

C. `idade_autor`

D. `"idade do autor"`

Isto é condição?

RESPOSTA

B. `idade_autor >= 60.`

Ela pergunta alguma coisa, e a resposta é VERDADEIRO ou FALSO. A A manda guardar, a C é só o nome de um campo e a D é um texto.

O que faltava em “boa quantidade”

FALHA 7, AULA 5

“Passe uma boa quantidade de Nutella.”

Vocês apontaram que não dá para verificar. Quem executa decide sozinho, e cada um decide diferente.

COM RÉGUA

`quantidade >= 1` colher de sopa

Dá para verificar, e duas pessoas verificam igual. A régua é um operador de comparação.

O pedido foi procedente

EXERCÍCIO

Queremos separar as decisões em que o pedido foi julgado procedente. Qual condição traduz isso?

A. `resultado = "PROCEDENTE"`

B. `resultado == "PROCEDENTE"`

C. `resultado != "PROCEDENTE"`

D. `resultado >= "PROCEDENTE"`

O pedido foi procedente

RESPOSTA

B. `resultado == "PROCEDENTE".`

Dois sinais de igual, e não um. A A manda guardar “PROCEDENTE” em `resultado`, e uma ordem de guardar não pergunta nada.

A D compara textos por grandeza, o que não quer dizer nada aqui: não existe um resultado maior que o outro.

Um sinal manda, o outro pergunta

=

Guarda.

```
idade_autor = 67
```

É uma ordem: passe a guardar 67 aqui.
Não devolve resposta nenhuma, porque não perguntou nada.

==

Pergunta.

```
idade_autor == 67
```

É uma pergunta: o que está guardado aqui é 67? Devolve VERDADEIRO ou FALSO.

Tudo menos um valor

EXERCÍCIO

Queremos separar as decisões em que o pedido não foi julgado improcedente, seja qual for o outro desfecho. Qual condição traduz isso?

A. `resultado == "IMPROCEDENTE"`

B. `resultado != "IMPROCEDENTE"`

C. `resultado == "PROCEDENTE"`

D. `resultado > "IMPROCEDENTE"`

Tudo menos um valor

RESPOSTA

B. `resultado != "IMPROCEDENTE".`

A C parece equivalente e não é: o campo tem quatro valores possíveis, então “não improcedente” alcança também o parcialmente procedente e o extinto sem resolução do mérito.

A D compara textos por grandeza, o que não quer dizer nada aqui: não existe um resultado maior que o outro.

Duas perguntas em sentido contrário

==
É este valor?

```
resultado == "IMPROCEDENTE"
```

VERDADEIRO só para as improcedentes, que são um dos quatro desfechos possíveis.

!=
É qualquer outro?

```
resultado != "IMPROCEDENTE"
```

VERDADEIRO para os outros três. Tudo, menos aquele valor.

As duas perguntam sobre o mesmo valor, e a resposta de uma é sempre o avesso da resposta da outra.

Qual operador

EXERCÍCIO

O Estatuto da Pessoa Idosa dá prioridade a quem tem idade igual ou superior a 60 anos. Qual condição traduz isso?

A. `idade_autor > 60`

B. `idade_autor >= 60`

C. `idade_autor = 60`

D. `idade_autor <= 60`

Qual operador

RESPOSTA

B. `idade_autor >= 60`.

A A deixaria de fora quem tem exatamente 60, que é justamente quem a lei quis incluir. Um caractere, e uma pessoa perde a prioridade.

Seis operadores de comparação

Operador	Lê-se	Exemplo no codebook
`==`	é igual a	`resultado == "PROCEDENTE"`
`!=`	é diferente de	`resultado != "IMPROCEDENTE"`
`>`	é maior que	`valor_causa > 10000.00`
`>=`	é maior ou igual a	`idade_autor >= 60`
`<`	é menor que	`idade_autor < 18`
`<=`	é menor ou igual a	`valor_causa <= 7850.50`

Toda comparação devolve VERDADEIRO ou FALSO. Nunca outra coisa.

Conectivos

Insper

Primeira

EXERCÍCIO

NÃO VERDADEIRO

dá VERDADEIRO ou FALSO?

A. VERDADEIRO	B. FALSO

Primeira

RESPOSTA

FALSO

O NÃO inverte, e só.

NÃO inverte, e só

NÃO VERDADEIRO é FALSO, e NÃO FALSO é VERDADEIRO.

A	NÃO A
V	F
F	V

Uma entrada só. Ele não junta duas condições, ele vira uma do avesso.

NÃO diante de NULO

tutela_deferida	NÃO tutela_deferida
VERDADEIRO	FALSO
FALSO	VERDADEIRO
NULO	VERDADEIRO

No campo `tutela_deferida`, `NULO` quer dizer que não houve pedido. O `NÃO` só devolve VERDADEIRO ou FALSO, e diante de `NULO` dá VERDADEIRO: `NÃO tutela_deferida` traz junto quem não pediu. O pedido indeferido se procura por `tutela_deferida == FALSO`.

Segunda

EXERCÍCIO

VERDADEIRO OU FALSO

dá VERDADEIRO ou FALSO?

A. VERDADEIRO	B. FALSO

Segunda

RESPOSTA

VERDADEIRO

Ao OU basta um lado.

OU: basta um dos dois

A	B	A OU B
V	V	V
V	F	V
F	V	V
F	F	F

Repare na primeira linha: com os dois verdadeiros o resultado ainda é VERDADEIRO. Este OU admite os dois ao mesmo tempo, diferente do “ou” do português corrente em “café ou chá”.

Reescrevam com OU

Lá atrás vocês viram estas duas condicionais separadas:

```
SE a parte de dentro do pão estiver coberta ENTÃO
    parar de passar Nutella
SE a Nutella acabar ENTÃO
    parar de passar Nutella
```

Escrevam isso numa condicional só, usando `OU`.

Reescrevam com OU

RESPOSTA

```
SE a parte de dentro do pão estiver coberta
   OU a Nutella acabar ENTÃO
    parar de passar Nutella
```

Duas condicionais que faziam a mesma coisa viram uma, e a instrução aparece uma vez só. Se amanhã ela mudar, muda num lugar em vez de dois.

Isto só funciona porque as duas levavam à mesma instrução. Se cada uma fizesse coisa diferente, elas continuariam separadas.

Terceira

EXERCÍCIO

VERDADEIRO E FALSO

dá VERDADEIRO ou FALSO?

A. VERDADEIRO	B. FALSO

Terceira

RESPOSTA

FALSO

O E exige os dois lados. Um lado falso derruba tudo.

E: exige os dois

A	B	A E B
V	V	V
V	F	F
F	V	F
F	F	F

Uma única linha devolve VERDADEIRO, e é a de cima. Basta um dos dois lados falhar para o conjunto inteiro dar FALSO.

Reescrevam com E

Este é o SE dentro do SE que vocês já viram, agora sem a linha `fechar a janela`:

```
SE estiver chovendo ENTÃO
    SE eu tiver guarda-chuva ENTÃO
        pegar o guarda-chuva
```

Escrevam isso numa condicional só, usando `E`.

Reescrevam com E

RESPOSTA

```
SE estiver chovendo E eu tiver guarda-chuva ENTÃO
    pegar o guarda-chuva
```

Dois níveis de recuo viraram um. O `E` exige os dois lados, e era isso que o aninhamento dizia: para pegar o guarda-chuva, precisava estar chovendo e eu ter guarda-chuva.

Quando aninhar e quando usar E

A reescrita funcionou porque a única coisa que aquele código fazia era a instrução mais interna. Devolvam a linha que estava lá antes:

```
SE estiver chovendo ENTÃO
    fechar a janela
    SE eu tiver guarda-chuva ENTÃO
        pegar o guarda-chuva
```

Agora o `E` não serve. Fechar a janela depende só da chuva, e dentro de um `E` ela passaria a depender também do guarda-chuva. Aninhar ganha exatamente quando existe alguma coisa assim.

As mesmas três palavras, em Python

Nos slides	Em Python	Exemplo
NÃO	`not`	`not tutela_deferida`
OU	`or`	`resultado == "PROCEDENTE" or resultado == "PARCIALMENTE PROCEDENTE"`
E	`and`	`idade_autor >= 60 and tutela_deferida`

Senão

O caso que a receita não previu

Na aula 5, a receita não dizia o que fazer se a Nutella acabasse antes de a parte de dentro do pão ficar coberta. Quem executou ao pé da letra parou, no meio, sem sanduíche.

```
SE ainda houver Nutella ENTÃO
    passar Nutella até cobrir a parte de dentro do pão
```

Este SE cobre um dos dois casos. Escrevam o outro.

O caso que a receita não previu

RESPOSTA

Duas condicionais separadas resolvem, e quase todo mundo começa assim:

```
SE ainda houver Nutella ENTÃO
    passar Nutella até cobrir a parte de dentro do pão
SE não houver mais Nutella ENTÃO
    parar de passar Nutella e fechar o sanduíche
```

Funciona, e escreve a mesma condição duas vezes, a segunda ao contrário: se uma mudar e a outra não, elas passam a discordar sem aviso.

Senão

Em Python: `else`

O caminho executado exatamente quando a condição do SE deu FALSO. Ele não tem pergunta própria: a pergunta é a mesma, e ele fica com tudo que sobrou dela.

EXEMPLO

```
SE ainda houver Nutella ENTÃO
    passar Nutella até cobrir a parte de dentro do pão
SENÃO
    parar de passar Nutella e fechar o sanduíche assim mesmo
```

A Nutella acabou

```
SE ainda houver Nutella ENTÃO
    passar Nutella até cobrir a parte de dentro do pão
SENÃO
    parar de passar Nutella e fechar o sanduíche assim mesmo
lavar a faca
```

Caso: a Nutella acabou. Quais linhas rodam?

A Nutella acabou

RESPOSTA

Parar de passar Nutella, fechar o sanduíche e lavar a faca.

A condição deu FALSO, então o primeiro ramo é pulado inteiro e o SENÃO roda. Lavar a faca está fora dos dois, e roda de qualquer jeito.

Um dos dois ramos sempre roda, e nunca os dois. É isso que o SENÃO garante e dois SE separados não garantem.

Marquem todas as decisões

O art. 71 do Estatuto da Pessoa Idosa dá prioridade na tramitação a quem tem 60 anos ou mais. Queremos que toda decisão da base receba uma marcação, e não só as prioritárias.

Escrevam isso com SE e SENÃO, usando o campo `idade_autor`.

Marquem todas as decisões

RESPOSTA

```
SE idade_autor >= 60 ENTÃO
    tramitacao = "prioritária"
SENÃO
    tramitacao = "comum"
```

Sem o SENÃO, as decisões de autores com menos de 60 anos ficariam sem marcação, indistinguíveis das que ninguém examinou.

Escrever `SE idade_autor < 60` no lugar do SENÃO também funciona, e custa uma segunda régua que alguém precisa lembrar de mudar junto com a primeira.

Quando o SENÃO faz falta

No lacre ele não fazia falta. Sem lacre não havia nada a fazer, e pular o bloco já era a resposta certa.

Aqui há o que fazer nos dois casos, e é isso que muda. O SENÃO cobre exatamente o que sobrou da pergunta, sem que ninguém precise escrevê-la de novo ao contrário.

A condição aparece uma vez só, e é impossível as duas metades discordarem.

Quando as hipóteses se sobrepõem

Insper

Prioridade, e prioridade dentro da prioridade

Estatuto da Pessoa Idosa. Art. 71.

> **Art. 71.** É assegurada prioridade na tramitação dos processos (...) em que figure como parte ou interveniente pessoa com idade igual ou superior a 60 (sessenta) anos, em qualquer instância.
>
> **§ 5º** Dentre os processos de pessoas idosas, dar-se-á prioridade especial aos das maiores de 80 (oitenta) anos.

Quem tem 84 anos preenche as duas hipóteses ao mesmo tempo. Não é erro do legislador: o § 5º ordena *dentro* da fila que o caput criou.

Como é que se escreve isso sem que uma das duas se perca?

Senão se

Em Python: `elif`

Testa outra condição, mas só quando a anterior deu FALSO. O SENÃO final recolhe todos os casos que nenhuma das condições pegou.

EXEMPLO

```
SE idade_autor >= 80 ENTÃO
    prioridade especial
SENÃO SE idade_autor >= 60 ENTÃO
    prioridade
SENÃO
    tramitação comum
```

Dois autores, o mesmo código

```
SE idade_autor >= 80 ENTÃO
    prioridade especial
SENÃO SE idade_autor >= 60 ENTÃO
    prioridade
SENÃO
    tramitação comum
```

O que cada um recebe: um autor de 65 anos e um autor de 30?

Dois autores, o mesmo código

RESPOSTA

65 ANOS

Prioridade.

A primeira condição deu FALSO, e é só por isso que a segunda chegou a ser testada.

30 ANOS

Tramitação comum.

As duas deram FALSO, e o SENÃO final recolhe o caso. Sem ele, este autor não receberia marcação nenhuma.

Exatamente um dos três caminhos roda, sempre. É isso que três testes soltos não garantem.

Autor de 84 anos: o que sai de cada lado?

VERSÃO 1

```
SE idade_autor >= 80 ENTÃO
    prioridade especial
SENÃO SE idade_autor >= 60
ENTÃO
    prioridade
```

VERSÃO 2

```
SE idade_autor >= 60 ENTÃO
    prioridade
SENÃO SE idade_autor >= 80
ENTÃO
    prioridade especial
```

As mesmas duas condições, a ordem trocada. O que cada versão devolve para um autor de 84 anos?

A ordem decide o resultado

RESPOSTA

VERSÃO 1

Prioridade especial.

A primeira condição pega o caso, e é a correta.

VERSÃO 2

Prioridade comum.

Quem tem 84 já satisfez `>= 60`, então o segundo teste nunca roda.

Mesmas duas condições, mesma lei, resultado oposto. Com `elif`, a ordem é conteúdo.

E com dois SE separados?

```
SE idade_autor >= 80 ENTÃO
    prioridade = "especial"
SE idade_autor >= 60 ENTÃO
    prioridade = "comum"
```

Sem `SENÃO SE`, dois `SE` independentes. Autor de 84 anos: o que fica guardado em `prioridade` no fim?

E com dois SE separados?

RESPOSTA

`"comum"`, e a prioridade especial se perdeu.

Os dois testes rodam sempre, porque o segundo `SE` não sabe que o primeiro já resolveu o caso. E como os dois gravam na mesma variável, a segunda escrita apaga a primeira.

Nem sempre isso é erro. O art. 1.048, I do CPC dá prioridade a quem tem 60 anos *ou* doença grave, sem graduar entre as duas. Ali, dois `SE` independentes é a tradução certa, desde que cada um grave no seu próprio campo.